

保護軟體供應鏈

來源：<https://codelabs.developers.google.com/securing-software-supply?hl=zh-tw>

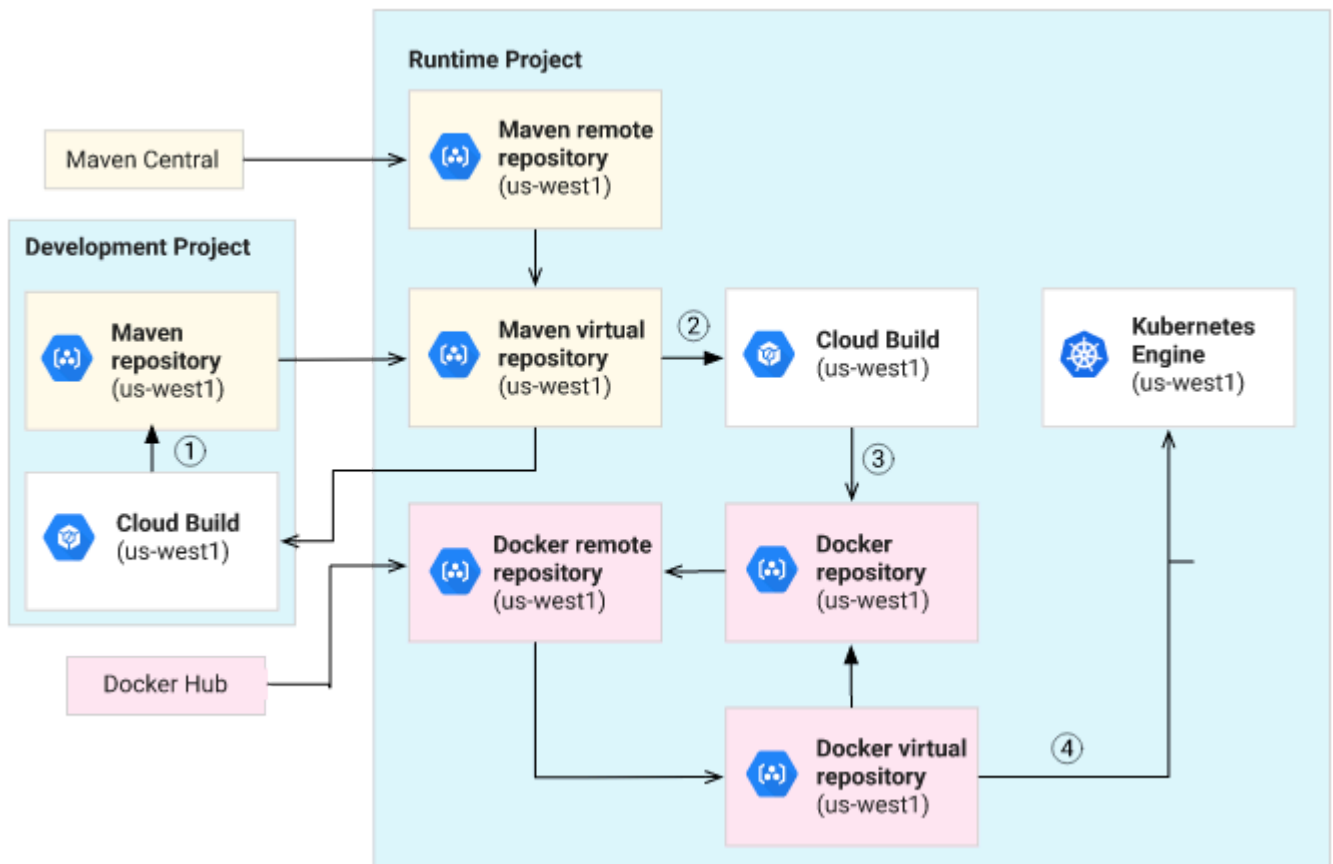
步驟數：5

目錄

1. [1. 總覽](#)
2. [2. 標準存放區](#)
3. [3. 遠端存放區](#)
4. [4. 虛擬存放區](#)
5. [5. 恭喜！](#)

1. 總覽

Artifact Registry 可讓您儲存不同類型的構件、在單一專案中建立多個存放區，並將特定區域或多個區域與每個存放區建立關聯。系統提供多種存放區模式，每種模式的用途各異。下圖展示了存放區多種應用方式之一，說明如何搭配使用不同模式的存放區，當中的工作流程橫跨兩個 Google Cloud 專案。在開發專案中，開發人員會建構 Java 應用程式。在另一個執行階段專案中，另一個建構作業會建立含有應用程式的容器映像檔，用於部署到 Google Kubernetes Engine。

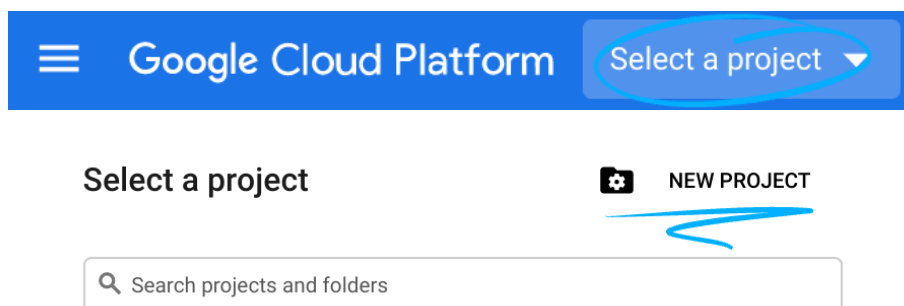


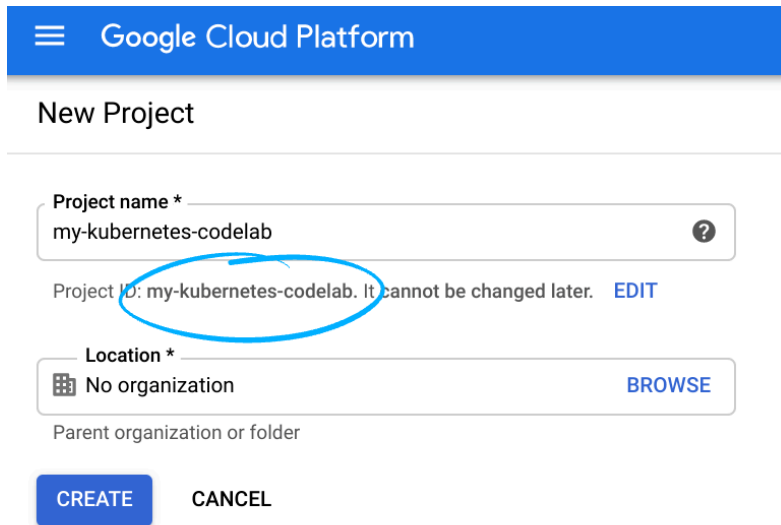
在本實驗室中，您將瞭解如何執行下列工作：

- 使用標準存放區部署私人套件
- 使用遠端存放區快取 Maven Central 套件
- 使用虛擬存放區，將多個上游存放區合併到一個設定中

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。





- 專案名稱是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新該位置資訊。
- 專案 ID 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生不重複的字串，通常您不需要在意這個字串。在大多數程式碼研究室中，您需要參照專案 ID (通常會標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間都會維持這個設定。
- 請注意，部分 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是專屬 ID，選定後就無法變更，且其他使用者無法使用。您是該 ID 的唯一使用者。即使刪除專案，該 ID 也無法再使用。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

- 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成本程式碼研究室的費用應該不高，甚至完全免費。如要關閉資源，避免產生本教學課程以外的費用，您可以刪除自己建立的資源，或刪除整個專案。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

Workspace 設定

設定 gcloud

在 Cloud Shell 設定專案 ID 和專案編號，分別儲存為 `PROJECT_ID` 和 `PROJECT_NUMBER` 變數。

〇〇〇〇

```
export PROJECT_ID=$(gcloud config get-value project)
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format='value(projectNumber)')
```

啟用 API

〇〇〇〇

```
gcloud services enable artifactregistry.googleapis.com
```

建立存放區的本機複本



```
git clone https://github.com/GoogleCloudPlatform/java-docs-samples  
cd java-docs-samples/container-registry/container-analysis
```

步驟 2 · 約 0 分鐘

2. 標準存放區

[標準存放區](#)可儲存私人套件，並在其他應用程式之間共用

建立標準 Maven 存放區

在 Cloud Shell 執行下列指令，為 Java 構件建立存放區：

```
gcloud artifacts repositories create container-dev-java-repo \
  --repository-format=maven \
  --location=us-central1 \
  --description="Java package repository for Container Dev Workshop"
```

如果出現 Cloud Shell 授權提示，請點選「授權」。

前往 Google Cloud 控制台的「Artifact Registry」-「Repositories」，就會看到新建立的 Maven 存放區 `container-dev-java-repo`。點選後，會發現存放區目前是空的。

```
gcloud artifacts repositories describe container-dev-java-repo \
  --location=us-central1
```

您應該會看到類似以下回應

```
Encryption: Google-managed key
Repository Size: 0.000MB
createTime: '2023-03-21T19:01:45.461589Z'
description: Java package repository for Container Dev Workshop
format: MAVEN
mavenConfig: {}
mode: STANDARD_REPOSITORY
name: projects/qwiklabs-gcp-03-4304110dc461/locations/us-central1/repositories/container-dev-java-repo
updateTime: '2023-03-21T19:01:45.461589Z'
```

設定 Maven 以搭配 Artifact Registry 使用

執行下列指令，顯示要新增至 Java 專案的存放區設定：

```
gcloud artifacts print-settings mvn \
  --repository=container-dev-java-repo \
  --location=us-central1
```

先前的指令會傳回要新增至專案 pom.xml 的 xml。

- 「repositories」區段會指定 Maven 可從何處下載遠端構件，供目前專案使用。
- 「distributionManagement」區段會指定專案部署時要推送至哪個遠端存放區。
- 「extensions」區段會新增 artifactregistry-maven-wagon，啟用連線至 Artifact Registry 所需的驗證和傳輸層

- 注意：擴充功能可放在 pom.xml 或 extensions.xml 中。如果專案依附於上層專案，系統會先存取這些依附元件，再載入 pom.xml 中的其餘項目。為確保上層能存取擴充功能，可以將擴充功能放在 extensions.xml 檔案中，這個檔案會在 pom.xml 之前載入，讓上層依附元件可以使用擴充功能。

複製這三個區段，然後在 Cloud Shell 編輯器中開啟 pom.xml，將傳回的設定新增至檔案底部，也就是結尾 `project` 標記內。

提示：在 Cloud Shell 中，於終端機執行下列指令，即可在目前目錄中開啟編輯器。

```
cloudshell workspace .
```

範例如下 (網址中的專案名稱會有所不同)：

```

...

<distributionManagement>
  <snapshotRepository>
    <id>artifact-registry</id>
    <url>artifactregistry://us-central1-maven.pkg.dev/qwiklabs-gcp-04-3c51830ea757/container-dev-java-repo</url>
  </snapshotRepository>
  <repository>
    <id>artifact-registry</id>
    <url>artifactregistry://us-central1-maven.pkg.dev/qwiklabs-gcp-04-3c51830ea757/container-dev-java-repo</url>
  </repository>
</distributionManagement>

<repositories>
  <repository>
    <id>artifact-registry</id>
    <url>artifactregistry://us-central1-maven.pkg.dev/qwiklabs-gcp-04-3c51830ea757/container-dev-java-repo</url>
    <releases>
      <enabled>true</enabled>
    </releases>
    <snapshots>
      <enabled>true</enabled>
    </snapshots>
  </repository>
</repositories>

<build>
  <extensions>
    <extension>
      <groupId>com.google.cloud.artifactregistry</groupId>
      <artifactId>artifactregistry-maven-wagon</artifactId>
      <version>2.2.0</version>
    </extension>
  </extensions>
</build>

</project>

```

將 Java 套件上傳至 Artifact Registry

在 Maven 設定 Artifact Registry 後，即可使用 Artifact Registry 儲存 Java JAR，供貴組織的其他專案使用。

執行下列指令，將 Java 套件上傳至 Artifact Registry：

```
mvn deploy -DskipTests
```

如要再次執行這項指令，請務必增加 pom.xml 中的版本號碼。

在 Artifact Registry 檢查 Java 套件

前往 [Cloud 控制台 - Artifact Registry - 存放區](#)，然後點按 `container-dev-java-repo`，確認 `hello-world` 二進位檔構件已存在：

Google Cloud

qwiklabs-gcp-04-3c51830ea757

Search (/) for res...

Search

S

Artifact Registry

Repositories

Settings

Release Notes

<|

Packages for ...

DELETE

EDIT REPOSITORY

SETUP INSTRUCTIONS

qwiklabs-gcp-04-3c51830ea757 > us-central1 > container-dev-java-repo

Repository Details

Format

Maven

Type

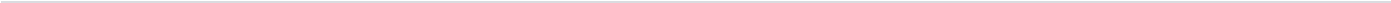
Standard

SHOW MORE

Filter

Enter property name or value

☐	Name ↑	Created	Updated
☐	com.google.containerregistry.containeranalysis	Just now	Just now



步驟 3 · 約 0 分鐘

3. 遠端存放區

[遠端存放區](#)可快取第三方套件，提高可靠性和安全性。

建立遠端存放區

注意：如要進一步瞭解驗證和設定，請參閱[產品說明文件](#)。

在 Cloud Shell 執行下列指令，為 Maven Central 構件建立遠端存放區：

```
gcloud artifacts repositories create maven-central-cache \
  --project=$PROJECT_ID \
  --repository-format=maven \
  --location=us-central1 \
  --description="Remote repository for Maven Central caching" \
  --mode=remote-repository \
  --remote-repo-config-desc="Maven Central" \
  --remote-mvn-repo=MAVEN-CENTRAL
```

在控制台中查看存放區

前往 [Cloud 控制台 - Artifact Registry - 存放區](#)，然後點選 `maven-central-cache`，會發現這個存放區已建立，但目前是空的。

在終端機中查看存放區

```
gcloud artifacts repositories describe maven-central-cache \
  --location=us-central1
```

將存放區整合至專案

執行下列指令，顯示要新增至 Java 專案的存放區設定：

```
gcloud artifacts print-settings mvn \
  --repository=maven-central-cache \
  --location=us-central1
```

在 pom.xml 中新增「repository」區段。請勿從輸出內容複製外部 `<repositories>` 標記。

將新加入的存放區 ID 變更為「central」，確保每個存放區項目都有專屬 ID。

範例如下 (網址中的專案名稱會有所不同)：

...

```
<distributionManagement>
  <snapshotRepository>
    <id>artifact-registry</id>
    <url>artifactregistry://us-central1-maven.pkg.dev/qwiklabs-gcp-04-3c51830ea757/container-dev-java-repo</url>
  </snapshotRepository>
  <repository>
    <id>artifact-registry</id>
    <url>artifactregistry://us-central1-maven.pkg.dev/qwiklabs-gcp-04-3c51830ea757/container-dev-java-repo</url>
  </repository>
</distributionManagement>

<repositories>
  <repository>
    <id>artifact-registry</id>
    <url>artifactregistry://us-central1-maven.pkg.dev/qwiklabs-gcp-04-3c51830ea757/container-dev-java-repo</url>
    <releases>
      <enabled>true</enabled>
    </releases>
    <snapshots>
      <enabled>true</enabled>
    </snapshots>
  </repository>

  <repository>
    <id>central</id>
    <url>artifactregistry://us-central1-maven.pkg.dev/qwiklabs-gcp-04-3c51830ea757/maven-central-cache</url>
    <releases>
      <enabled>true</enabled>
    </releases>
    <snapshots>
      <enabled>true</enabled>
    </snapshots>
  </repository>

</repositories>

<build>
  <extensions>
    <extension>
      <groupId>com.google.cloud.artifactregistry</groupId>
      <artifactId>artifactregistry-maven-wagon</artifactId>
      <version>2.2.0</version>
    </extension>
  </extensions>
</build>
```

```
</project>
```

在終端機中執行下列指令，為專案建立 `extensions.xml`，使用[核心擴充功能](#)機制，確保 Maven 可從 Artifact Registry 解析上層或外掛程式依附元件。

```
mkdir .mvn
cat > .mvn/extensions.xml << EOF
<extensions xmlns="http://maven.apache.org/EXTENSIONS/1.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/EXTENSIONS/1.0.0
http://maven.apache.org/xsd/core-extensions-1.0.0.xsd">
  <extension>
    <groupId>com.google.cloud.artifactregistry</groupId>
    <artifactId>artifactregistry-maven-wagon</artifactId>
    <version>2.2.0</version>
  </extension>
</extensions>
EOF
```

從遠端存放區提取依附元件

執行下列指令，使用遠端存放區編譯應用程式：

```
rm -rf ~/.m2/repository
mvn compile
```

在控制台查看套件

前往 [Cloud 控制台 - Artifact Registry - 存放區](#)，然後點選 `maven-central-cache`，確認二進位檔構件已快取至此：

☐	Name 	Created	Updated
☐	com.github.spotbugs:spotbugs-annotations	1 minute ago	1 minute ago
☐	com.google.android:annotations	1 minute ago	1 minute ago
☐	com.google.api-client:google-api-client-bom	1 minute ago	1 minute ago
☐	com.google.api.grpc:proto-google-cloud-containeranalysis-v1	2 minutes ago	2 minutes ago
☐	com.google.api.grpc:proto-google-cloud-containeranalysis-v1beta1	1 minute ago	1 minute ago
☐	com.google.api.grpc:proto-google-cloud-pubsub-v1	1 minute ago	1 minute ago
☐	com.google.api.grpc:proto-google-common-protos	2 minutes ago	2 minutes ago
☐	com.google.api.grpc:proto-google-iam-v1	1 minute ago	1 minute ago

步驟 4 · 約 0 分鐘

4. 虛擬存放區

[虛擬存放區](#)可做為多個存放區的介面，透過單一設定存取，簡化構件使用者的用戶端設定，並防範[依附元件混淆攻擊](#)，進一步保障安全。

建立政策檔案

```
cat > ./policy.json << EOF
[
  {
    "id": "private",
    "repository": "projects/${PROJECT_ID}/locations/us-central1/repositories/container-dev-
java-repo",
    "priority": 100
  },
  {
    "id": "central",
    "repository": "projects/${PROJECT_ID}/locations/us-central1/repositories/maven-central-
cache",
    "priority": 80
  }
]

EOF
```

建立虛擬存放區

```
gcloud artifacts repositories create virtual-maven-repo \
  --project=${PROJECT_ID} \
  --repository-format=maven \
  --mode=virtual-repository \
  --location=us-central1 \
  --description="Virtual Maven Repo" \
  --upstream-policy-file=./policy.json
```

將存放區整合至專案

執行下列指令，顯示要新增至 Java 專案的存放區設定：

```
gcloud artifacts print-settings mvn \
  --repository=virtual-maven-repo \
  --location=us-central1
```

將 pom 中的整個「repositories」區段，替換為輸出內容中的「virtual repositories」區段。

範例如下 (網址中的專案名稱會有所不同)：

```

...

<distributionManagement>
  <snapshotRepository>
    <id>artifact-registry</id>
    <url>artifactregistry://us-central1-maven.pkg.dev/qwiklabs-gcp-04-3c51830ea757/container-dev-java-repo</url>
  </snapshotRepository>
  <repository>
    <id>artifact-registry</id>
    <url>artifactregistry://us-central1-maven.pkg.dev/qwiklabs-gcp-04-3c51830ea757/container-dev-java-repo</url>
  </repository>
</distributionManagement>

<repositories>
  <repository>
    <id>artifact-registry</id>
    <url>artifactregistry://us-central1-maven.pkg.dev/qwiklabs-gcp-04-3c51830ea757/virtual-maven-repo</url>
    <releases>
      <enabled>true</enabled>
    </releases>
    <snapshots>
      <enabled>true</enabled>
    </snapshots>
  </repository>
</repositories>

<build>
  <extensions>
    <extension>
      <groupId>com.google.cloud.artifactregistry</groupId>
      <artifactId>artifactregistry-maven-wagon</artifactId>
      <version>2.2.0</version>
    </extension>
  </extensions>
</build>

</project>

```

從虛擬存放區提取依附元件

由於虛擬存放區是傳遞路徑，不會儲存任何實際套件，為了清楚示範程序，您將刪除先前建立的 maven-central-cache 存放區並重新建立，使用空的存放區重新開始

執行下列指令，重新建立快取存放區

```
gcloud artifacts repositories delete maven-central-cache \
  --project=$PROJECT_ID \
  --location=us-central1 \
  --quiet

gcloud artifacts repositories create maven-central-cache \
  --project=$PROJECT_ID \
  --repository-format=maven \
  --location=us-central1 \
  --description="Remote repository for Maven Central caching" \
  --mode=remote-repository \
  --remote-repo-config-desc="Maven Central" \
  --remote-mvn-repo=MAVEN-CENTRAL
```

您可以在控制台查看空的存放區，方式如下：前往 [Cloud 控制台 - Artifact Registry - 存放區](#)

現在，請使用下列指令建構專案，以操作虛擬存放區：

```
rm -rf ~/.m2/repository
mvn compile
```

在控制台查看套件

前往 [Cloud 控制台 - Artifact Registry - 存放區](#)。點選 `maven-central-cache`，確認二進位檔構件已設定為從虛擬存放區提取，但最終是從 `maven-central-cache` 提取：

The screenshot shows the Google Cloud Artifact Registry interface. The top navigation bar includes the Google Cloud logo, a project selector (qwiklabs-gcp-04-a8b4ca853bcf), a search bar, and various utility icons. The left sidebar contains the 'Artifact Registry' menu with options for 'Repositories', 'Settings', and 'Release Notes'. The main content area is titled 'Packages for ...' and shows the breadcrumb path: 'qwiklabs-gcp-04-a8b4ca853bcf > us-central1 > maven-central-cache'. Below this, the 'Repository Details' section shows 'Format: Maven' and 'Type: Remote'. A 'SHOW MORE' link is available. The bottom section, titled 'Filter', displays a table of packages with columns for 'Name', 'Created', and 'Updated'. The table lists several packages, including 'com.github.spotbugs:spotbugs-annotations' and 'com.google.android:annotations', all of which were created and updated '1 minute ago' or '2 minutes ago'.

Name	Created	Updated
com.github.spotbugs:spotbugs-annotations	1 minute ago	1 minute ago
com.google.android:annotations	1 minute ago	1 minute ago
com.google.api-client:google-api-client-bom	1 minute ago	1 minute ago
com.google.api.grpc:proto-google-cloud-containeranalysis-v1	2 minutes ago	2 minutes ago
com.google.api.grpc:proto-google-cloud-containeranalysis-v1beta1	1 minute ago	1 minute ago
com.google.api.grpc:proto-google-cloud-pubsub-v1	1 minute ago	1 minute ago
com.google.api.grpc:proto-google-common-protos	2 minutes ago	2 minutes ago
com.google.api.grpc:proto-google-iam-v1	1 minute ago	1 minute ago

步驟 5 · 約 0 分鐘

5. 恭喜！

恭喜，您已完成本程式碼研究室！

涵蓋內容

- 使用標準存放區部署私人套件
- 使用遠端存放區快取 Maven Central 套件
- 使用虛擬存放區，將多個上游存放區合併到一個設定中

清除所用資源

執行下列指令即可刪除專案：

```
gcloud projects delete ${PROJECT_ID}
```

上次更新時間：2023 年 3 月 22 日
